

Design Documentation

High-Level Design

Instacart Market Basket Analysis — Python, MySQL Database, and Tableau Pipeline

This document describes the architecture and implementation detail of the end-to-end analytics pipeline built for the Instacart Market Basket dataset. It is organized into two parts: the High-Level Design (HLD), which describes system architecture and component-level responsibilities, and the Low-Level Design (LLD), which describes the detailed process flow, folder structure, and module-level implementation.

Part I — High-Level Design (HLD)

1. Purpose and Scope

This project ingests raw grocery order data, benchmarks two data-access strategies, performs exploratory data analysis in Python, and produces business-facing dashboards and a final report. The scope covers data ingestion, cleaning, analysis, visualization, and reporting.

2. Architecture Overview

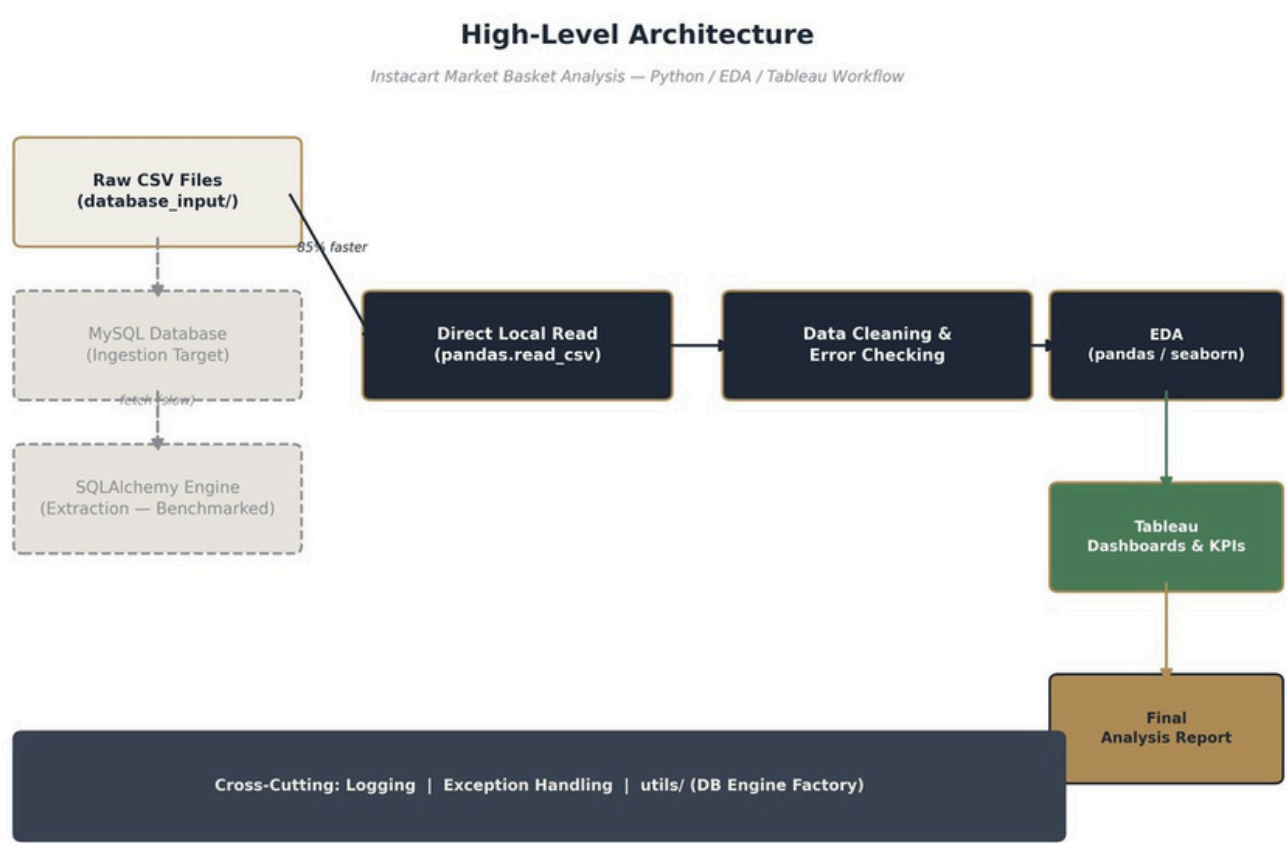


Figure 1. High-level system architecture and data flow.

Raw CSV files are placed in a local input directory. A MySQL ingestion path was built using SQLAlchemy, but a direct-read benchmark showed that reading CSVs from disk with pandas is approximately 85% faster than fetching the same data back out of MySQL. As a result, the production analysis path reads directly from the local files, while the database ingestion and engine components remain in place as a demonstrated, reusable capability rather than the active data path.

3. Components

Component	Responsibility
Ingestion Layer	Loads raw CSVs into MySQL via a SQLAlchemy-managed engine; retained for demonstration and future use.
Data Access Layer	Reads source CSVs directly from local storage for all active analysis (chosen path).
Processing Layer	Cleans, validates, and merges datasets; performs exploratory data analysis in Python (pandas, seaborn).
Visualization Layer	Tableau dashboards presenting KPIs and segment-level views derived from the cleaned dataset.
Reporting Layer	Consolidated business report combining EDA insights and dashboard findings.
Cross-Cutting Utilities	Logging and exception handling used throughout ingestion and processing steps; a reusable database engine factory in utils/.

4. Technology Stack

Layer	Technology
Language	Python 3
Data Manipulation	pandas
Visualization (Python)	seaborn, matplotlib
Database	MySQL
Database Connectivity	SQLAlchemy
Business Intelligence	Tableau
Development Environment	VS Code / Jupyter Notebook

5. Key Design Decision: Data Access Strategy

A deliberate architectural decision was made after benchmarking two approaches to accessing the source data:

Approach	Outcome
Fetch via SQLAlchemy from MySQL	Functional, but measurably slower for repeated analysis workloads.
Direct read from local CSVs (pandas.read_csv)	Approximately 85% faster; adopted as the active data access path for all EDA work.

The MySQL ingestion and SQLAlchemy engine components were intentionally kept in the codebase. They demonstrate a database-backed pipeline pattern and remain available if the project scales to a context where centralized, queryable storage is required.

6. High-Level Data Flow Summary

- Raw CSVs are staged in a local input directory.
- Data is optionally ingested into MySQL through a reusable engine factory (utils/).
- Active analysis reads source files directly from local storage.
- Cleaned, merged data flows into a Python EDA notebook, organized into six thematic sections.
- Key findings are visualized in Tableau as KPI cards and segmentation dashboards.
- EDA insights and dashboard findings are consolidated into a final business report.